Advanced Engineering Informatics 76 (2026) 105162

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Full length article

FedOut: Federated learning with outsourced homomorphic encryption for resource-constrained IoT devices

Yi Yuan [a], Hong Rao [a,*], Shuanggen Liu [b,*], Rixuan Qiu [c], Mengyuan Cui [b], Yuxin Hu [b]

[a] School of Software, Nanchang University, Nanchang 330000, China
[b] School of Cyberspace Security, Xi'an University of Posts and Telecommunications, Xi'an 710121, China
[c] Information and Communication Branch, State Grid Jiangxi Electric Power Company, Nanchang 330077, China

ARTICLE INFO

Keywords:
Federated learning
Privacy preservation
Secure outsourcing
Homomorphic encryption

ABSTRACT

Federated learning (FL) preserves data privacy by keeping raw data local, yet plaintext model updates remain vulnerable to inference attacks. Homomorphic encryption (HE) protects updates through ciphertext aggregation, but single-key HE risks data exposure among clients and imposes heavy computational and communication overhead on resource-constrained IoT devices. Guided by three key research questions, this paper proposes FedOut, an efficient FL framework that combines an enhanced multi-key CKKS scheme with a secure outsourcing protocol SOP. The multi-key scheme supports efficient single-step decryption, while SOP provides client dropout tolerance and offloads costly encryption to a proxy. Formal security analysis confirms the privacy guarantees of FedOut, and experiments on real datasets demonstrate its minimal client-side computational latency and robust aggregation under client dropout, validating the resolution of the three research questions.

1. Introduction

In recent years, the number of IoT devices has grown explosively. Due to their distributed nature and environmental awareness, using the data collected or generated by IoT devices for distributed machine learning has become an active research area. However, IoT data may contain sensitive information, such as medical records and financial details, thus necessitating privacy-preserving measures to prevent the leakage of such sensitive data. Federated learning (FL) is an emerging machine learning paradigm that was proposed in 2016 [1], which allows clients to train models locally and share only model updates. In this way, clients' private data remains stored locally at all times. However, existing research [2,3] has shown that sharing plaintext model updates remains vulnerable, which allows adversaries to infer sensitive information.

To protect privacy in federated learning, several privacy-preserving techniques have been proposed, primarily including secure aggregation (SA), secure multiparty computation (SMC), differential privacy (DP), and homomorphic encryption (HE). SA is a privacy-preserving technique that protects individual client updates by applying random masks that cancel out during aggregation. However, the aggregation server still learns the aggregated model updates. Furthermore, each training round requires clients to synchronize keys and exchange mask materials [4,5], which imposes significant communication overhead on the clients. SMC is an advanced cryptographic technique that enables multiple mutually distrusting parties to jointly perform a computation without exposing their private input data. However, this approach introduces substantial communication overhead and often requires careful adaptation of machine learning algorithms to fit its framework [6,7]. DP protects privacy by injecting calibrated noise into the local updates before they are uploaded. While effective to a degree, this noise inevitably compromises the accuracy of model training [8,9]. More importantly, DP-based mechanisms generally offer a weaker security guarantee and may still entail risks of potential privacy leakage [10]. In contrast, HE allows each client to encrypt updates locally before uploading, and the server then performs aggregation directly on the ciphertexts [11,12]. From an attacker's perspective, the updates remain encrypted throughout, making it infeasible to infer any private information without the decryption key, thus providing a high level of security. Furthermore, HE can be integrated into federated learning with relative ease, without requiring major modifications to the existing FL framework.

Nevertheless, the application of HE in federated learning still faces two major challenges. First, in conventional single-key HE schemes, all clients share the same decryption key. This implies that any client could intercept and decrypt the ciphertexts of others. Therefore, in

* Corresponding authors.
E-mail addresses: raohong@ncu.edu.cn (H. Rao), liushuanggen201@xupt.edu.cn (S. Liu).

https://doi.org/10.1016/j.aei.2026.105162
Received 24 February 2026; Received in revised form 6 August 2026; Accepted 13 August 2026
Available online 21 August 2026
1474-0346/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

this case, multi-key homomorphic encryption schemes, in which each client encrypts updates with its own key, present a natural and effective solution. While some multi-key solutions have been developed, most fail to adequately handle client dropout, which can lead to decryption failures [13,14], and it is worth noting that client dropout occurs particularly frequently in certain scenarios, such as agricultural IoT devices operating in remote farmland or mountainous areas with poor signal conditions. Besides, existing multi-key HE schemes often require additional interaction rounds during decryption [13,15], further increasing communication costs. Therefore, it is necessary to introduce proper recovery mechanisms for dropout clients and to simplify the decryption process. Second, HE inherently incurs significant computational and communication overhead. Specifically, operations on ciphertext are far more complex than on plaintext due to operations such as large-scale polynomial modular multiplication, which are extremely time-consuming. As shown in Fig. 1, we executed the CKKS benchmark program in the SEAL library [16] and verified that encryption is the operation with the highest computational overhead among all operations. Furthermore, the size of the ciphertext can be dozens or even hundreds of times larger than the original data after encryption [11,17]. This is particularly challenging for devices with limited computational resources and network bandwidth, making it crucial to explore ways to reduce these high costs.

Based on the above analysis, we identify three **research gaps** that have not been adequately addressed: (1) few existing multi-key HE schemes provide non-interactive local decryption; (2) no generic and secure outsourcing protocol exists for RLWE-based encryption that is compatible with the non-streaming, per-round encryption pattern of FL; (3) a practical software-based dropout recovery mechanism that avoids per-round re-keying or trusted hardware has not been integrated into an HE-based FL framework.

To bridge these gaps, we propose three **research questions (RQs)**, which are aligned with Gaps (1)–(3) respectively:

- **RQ1** (Non-interactive decryption): How can multi-key HE be redesigned to support single-step local decryption without any client interaction?
- **RQ2** (Secure offloading): How can the RLWE-based HE encryption workload be securely offloaded to a proxy server, so that clients only perform lightweight operations?
- **RQ3** (Dropout resilience): How can the FL framework maintain correct decryption and stable model convergence when clients drop out arbitrarily, without re-keying or trusted hardware?

Contributions. Our contributions are organized around the above research questions. Specifically:

- To answer RQ1, we present ECMK-CKKS, which improves xMK-CKKS [13] by eliminating the steps of calculating and uploading decryption shares. Thus, each client can perform decryption locally using a common decryption key.
- To answer RQ2 and RQ3 jointly, we design a secure outsourcing protocol SOP that delegates the heavy encryption operations to a high-performance proxy server and provides a dropout protection mechanism. Resource-constrained clients are only required to perform lightweight blinding operations.
- Integrating ECMK-CKKS and SOP, we build FedOut, a secure, robust, and efficient FL framework that is particularly friendly to resource-constrained IoT devices and effectively addresses all three research questions.
- Through formal security analysis that establishes the security of FedOut and extensive experiments that confirm its efficiency and robustness, we validate that our design conclusively addresses the three research questions.

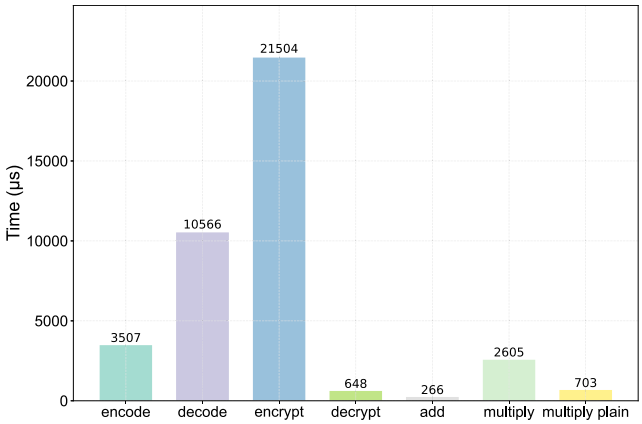


Fig. 1. Breakdown of computational cost in HE.

Organization. The remainder of this paper is organized as follows. In Section 2, we provide a brief review of the related work. In Section 3, we introduce the preliminaries used in this paper. Our proposed scheme FedOut is detailed in Section 4, followed by its security analysis in Section 5. In Section 6, we present a series of experiments to evaluate the performance of FedOut. In Section 7, we provide a comprehensive discussion of how each research question is addressed. Finally, we conclude our work in Section 8.

2. Related work

In this section, we first review multi-key HE schemes, highlighting their deficits in enabling non-interactive decryption (RQ1) and tolerating client dropout (RQ3). We then examine cryptographic outsourcing techniques, focusing on the lack of a generic and secure outsourcing protocol that can offload RLWE-based encryption in the non-streaming FL setting (RQ2).

2.1. FL based on multi-key HE

Although HE holds significant promise for privacy preservation in federated learning, traditional single-key HE schemes present notable security challenges in multi-client scenarios. A feasible solution to address these issues is to adopt multi-key HE schemes. We organize existing multi-key HE schemes into three categories and examine their respective limitations.

(1) Extending existing single-key HE schemes to support multiple keys. [15] proposed multi-key variants of CKKS [18] and BFV [19], namely MK-CKKS and MK-BFV. However, their distributed decryption requires interaction between clients and the server, thus failing to meet the non-interactive requirement of RQ1 and incurring extra communication latency. More critically, this scheme cannot be directly applied to federated learning, because the server can decrypt individual client ciphertexts, leading to potential privacy leakage. This limitation was addressed in [13], which improved MK-CKKS and introduced another variant called xMK-CKKS. Nevertheless, it still involves interactions in decryption and cannot tolerate client dropout, hence failing to meet the requirements of both RQ1 and RQ3. [20] presented EMK-BFV, a multi-key variant of BFV based on the Trusted Execution Environment (TEE) that offers a client dropout protection mechanism but relies on specialized hardware, which falls short of the software-only dropout resilience required by RQ3. Moreover, due to the frequent noise refreshing required during aggregation, it incurs substantial computational overhead.

(2) Designing new natively multi-key HE schemes. [17] designed a novel multi-key HE algorithm named ESHE, which aggregates all client

encryption keys into a fixed decryption key. The key enables all clients to perform decryption locally in a single step. However, because the decryption key remains static and equals the sum of all encryption keys, it becomes ineffective when client dropout occurs, thereby violating the dropout tolerance requirement of RQ3. DTAHE [21] proposed a decentralized threshold additive HE scheme, but its fixed threshold limits its ability to handle arbitrary client dropout, again leaving RQ3 unaddressed.

(3) Bridging single-key to multi-key HE via an intermediate primitive. [12] employed proxy re-encryption to convert ciphertexts under different client keys into ciphertexts under a unified server key, thereby supporting non-interactive decryption and tolerating client dropout. However, this approach is built on elliptic-curve encryption and does not extend to RLWE-based HE, which is the cryptographic foundation we adopt for its post-quantum security. Consequently, despite satisfying RQ1 and RQ3 in its own setting, it cannot be directly integrated into an FL framework that relies on outsourced RLWE-based encryption.

In summary, the reviewed schemes either require interactive decryption or prove fragile under client dropout, and the only approach that satisfies both relies on elliptic-curve cryptography incompatible with RLWE-based outsourcing. An RLWE-compatible multi-key HE scheme that simultaneously achieves non-interactive decryption and dropout resilience therefore remains absent, directly motivating our ECMK-CKKS design and the supplementary-ciphertext mechanism in SOP.

2.2. Secure outsourcing of cryptographic computations

Certain cryptographic operations typically incur substantial computational overhead, making them difficult to execute on resource-constrained IoT devices. Fortunately, with the advancement of cloud computing, outsourcing such complex cryptographic computations to proxy servers with powerful computational capabilities has emerged as a viable solution. Hohenberger and Lysyanskaya [22] introduced a formal definition for securely outsourcing computational tasks from resource-constrained clients to untrusted servers. Subsequently, a series of research has focused on how to securely outsource various intensive cryptographic computations to third parties, including outsourcing modular exponentiation [23,24], modular inversion [25,26], and bilinear pairing [27].

However, in homomorphic encryption based on Learning with Errors (LWE) or Ring Learning with Errors (RLWE), the primary computational bottleneck lies in modular multiplication operations involving high-dimensional matrices or large-scale polynomials. [28] designed an n-secret LWE-based homomorphic encryption scheme tailored for matrices that supports outsourcing of matrix multiplication. However, it cannot be easily and directly applied to existing homomorphic encryption schemes such as BFV and CKKS, where the plaintext is typically represented in polynomial form. In contrast, [29] focused on outsourcing cryptographic operations involving large-scale polynomial modular multiplication and proposed an outsourced decryption protocol for RLWE-based homomorphic encryption, yet its decryption protocol cannot be directly applied to encryption outsourcing. To the best of our knowledge, the most relevant work that targets the outsourcing of encryption for RLWE-based homomorphic encryption is [30]. Specifically, they proposed an outsourcing scheme for plaintext encryption under CKKS, but it is primarily designed for streaming data applications and cannot function effectively in non-streaming scenarios.

Therefore, a generic and secure outsourcing protocol that can offload RLWE-based encryption in the non-streaming, per-round FL setting remains absent, leaving RQ2 unaddressed.

3. Preliminaries

In this section, we first introduce the process of federated learning, then present homomorphic encryption technology. A basic understanding of these concepts is helpful for following how FedOut addresses the three research questions in the subsequent sections.

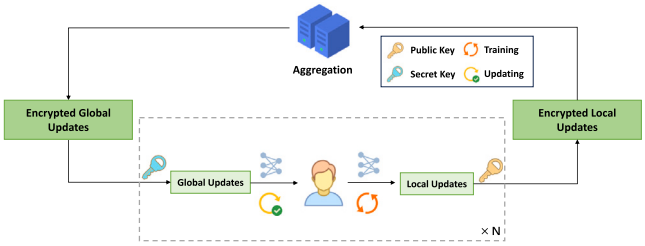


Fig. 2. Illustration of HE-based privacy preservation in FL.

3.1. Federated learning

Federated learning is a machine learning paradigm that enables multiple clients to collaborate in training a global model with the assistance of a server without disclosing local privacy. Consider that in a typical federated learning framework, there are N clients and one aggregation server. The specific training process is as follows:

(1) Initialization: The server sends the initial model w^0 to the clients participating in the training.

(2) Local training: Each client i uses the locally stored private dataset D_i to train the global shared model w^t for the tth round, and calculates the local gradient $g_i^t = \nabla_w L(D_i, w^t)$, where L represents the loss function. Then, clients will upload their local gradients to the server.

(3) Aggregation: The server receives all the local gradients of the clients, and then uses the federated average algorithm FedAvg [31] to aggregate these gradients, obtaining the global gradient $G^t = \sum_{i=1}^{N} \frac{|D_i|}{|D|} g_i^t$, where $|D_i|$ represents the size of the local dataset of client i, and $|D|$ represents the total size of all client datasets. The server then distributes G^t to each client.

(4) Model update: Each client updates the global model using the global gradient G^t, resulting in $w^{t+1} = w^t - \eta \cdot G^t$, where η is the learning rate.

During the federated learning training process, steps (2)–(4) will be repeated until the model converges or the specified number of rounds is reached.

3.2. Homomorphic encryption

Homomorphic encryption allows arithmetic operations to be directly performed on encrypted data without the need for decryption in advance, and the decryption results of ciphertext operations are consistent with those of plaintext operations. For a more formal explanation, let $Enc(\cdot, ek)$ denote the encryption algorithm under an encryption key ek, and $Dec(\cdot, dk)$ denote the decryption algorithm under the corresponding decryption key dk. For a set of plaintexts $\{m_1, m_2, \ldots, m_N\}$, an HE scheme is said to be additively homomorphic if there exists an operation $\oplus$ on ciphertexts such that

$$\mathrm{Dec}\Big(\bigoplus_{i=1}^{N} \mathrm{Enc}(m_i, ek), dk\Big) = \sum_{i=1}^{N} m_i \ , \tag{1}$$

i.e., the decryption of aggregated ciphertext equals the sum of the original plaintexts. Similarly, the scheme is multiplicatively homomorphic if there exists an operation $\otimes$ on ciphertexts satisfying

$$\mathrm{Dec}\Big(\bigotimes_{i=1}^{N} \mathrm{Enc}(m_i, ek), dk\Big) = \prod_{i=1}^{N} m_i \ . \tag{2}$$

The classic homomorphic encryption algorithm Paillier [32] only supports addition operations, while some homomorphic encryption algorithms, such as BFV [19] and CKKS [18], support both addition and multiplication operations simultaneously. Homomorphic encryption ensures strong privacy by enabling secure aggregation of encrypted

updates. For example, as illustrated in Fig. 2, a client can encrypt its local model updates using HE and upload the ciphertexts to the server. The server then aggregates the received ciphertexts and distributes the aggregated ciphertext back to the clients. This end-to-end encryption mechanism ensures that all intermediate results remain confidential, significantly reducing the risk of privacy leakage.

4. Proposed FedOut scheme

This section presents FedOut, our solution designed to systematically answer the three RQs posed in Section 1. We first give an overview of the framework, and then detail the enhanced multi-key HE scheme ECMK-CKKS, which serves as our answer to RQ1, followed by the secure outsourcing protocol SOP, where RQ2 and RQ3 are jointly addressed. Finally, we discuss real deployment challenges.

4.1. Overview

The proposed system consists of four types of entities: a key distribution center (KDC), an aggregation server (AS), a computational proxy server (Proxy CS), and a set of N clients. Among them, the KDC is responsible for distributing key pairs and public parameters to clients, outsourcing parameters to Proxy CS. The clients perform local model training and generate model updates, which are blinded and then sent to Proxy CS. The Proxy CS acts as a proxy that carries out homomorphic encryption and sends blinded ciphertexts to AS. The AS will report the dropout clients' IDs ($DropoutID$) to Proxy CS to obtain supplementary ciphertexts. Then, the AS deblinds and aggregates all ciphertexts and returns the aggregated result to clients participating in the current round.

Algorithm 1 Parameter Generation

Input: Security parameter λ.
Output: Public parameters PP, outsourcing parameters OP.
1: Choose an integer $n = 2^{\log n}$ as RLWE dimension.
2: Choose an integer $p = 2^{\log p}$ as scaling factor.
3: Choose a prime ciphertext modulus q with size $\log q$ bits.
4: Define the ternary key distribution χ.
5: Define the discrete Gaussian error distribution ψ.
6: Define the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$.
7: Uniformly sample polynomial $a \leftarrow \mathcal{U}(R_q)$ and sample $v \leftarrow \chi$.
8: **If** HE.outsourced = True:
9: Return $PP = \{n, p, q\}$.
10: Return $OP = \{n, q, \chi, \psi, a\}$.
11: **Else:**
12: Return $PP = \{n, p, q, \chi, \psi, a, v\}$.
13: **End if**.

Threat Model. In FedOut, the KDC is assumed to be fully trusted: it neither actively leaks clients' privacy nor conspires with other entities. This design is widely adopted in key distribution and management [11, 33]. Since the KDC participates only in the initialization phase to distribute parameters and key pairs, its online time is very short, and the probability of compromise is extremely low. In contrast, all other entities are considered honest-but-curious. That is, they faithfully follow the protocol but may attempt to learn private information from clients' data. We consider that clients may collude with the AS or the Proxy CS to extract other clients' private data. However, we do not consider collusion between the AS and the Proxy CS. This two-server non-collusion assumption is commonly adopted in existing privacy-preserving FL works or outsourced computing [34,35]. Communications between the KDC and all other entities, as well as communications between clients and the Proxy CS and between clients and the AS, are conducted over secure channels, while all other communications occur over public channels. We consider that entities other than the KDC may monitor communication channels to infer private data. The threat model is summarized in Table 1.

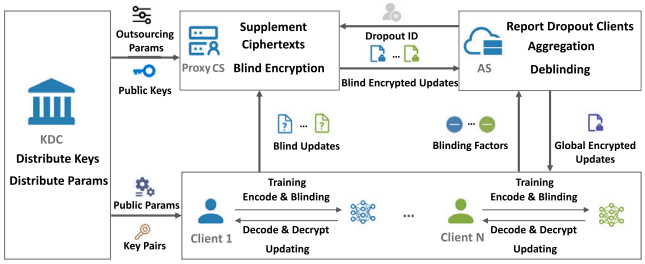


Fig. 3. FedOut system model.

System Model. FedOut is designed to have two modes: local mode and outsourced mode. In outsourced mode, SOP will replace the local encryption module in ECMK-CKKS to achieve the goal of transferring the client's computational burden and dropout protection mechanism. However, when SOP is unavailable, such as lacking a suitable third-party proxy, FedOut will degenerate into local mode, where the client will have to bear the encryption task itself and cannot achieve dropout protection. We illustrate our system model in Fig. 3. In the following, we describe the workflow of our solution under the outsourced mode, which is the primary operating mode of FedOut.

(1) *System Setup:* KDC generates key pairs $\{sk_i, pk_i, s\}$ for each client, as well as the public parameters PP required for blinding. These materials are distributed to each client via a mutually authenticated TLS channel established by the TLS/SSL protocol. Meanwhile, the outsourcing parameters OP are also sent to Proxy CS over a secure channel. Finally, KDC will also send the public key $\{pk_i\}_{i=1}^{N}$ of each client to Proxy CS via a public channel for subsequent outsourced encryption. The detailed procedures are elaborated in Algorithm 1 and Algorithm 2.

(2) *Local Training:* At the start of round t, each client i obtains the current global model w^t. It then trains this model using its local private dataset D_i. After performing E local training epochs, the client produces its local update g_i^t.

(3) *Encoding & Blinding:* Each client first encodes its local update g_i^t from a real-valued vector into a polynomial m_i^t over R using the CKKS encoding scheme. It then blinds the plaintext polynomial m_i^t with a blinding factor r_i^t to obtain the blinded plaintext $m_i^{t\prime}$. To prevent eavesdropping by other entities, client i transmits $m_i^{t\prime}$ to the Proxy CS and r_i^t to the AS via secure channels, respectively.

(4) *Blind Encryption:* Using the blinded plaintext $m_i^{t\prime}$ received from client i, the Proxy CS performs the blind encryption. The encryption algorithm employed is our enhanced multi-key homomorphic scheme ECMK-CKKS, which is detailed in Section 4.2. After completing the encryption, Proxy CS sends the resulting blinded ciphertext $c_i^{t\prime}$ to AS.

(5) *Deblinding & Supplement ciphertexts:* After receiving all blinded ciphertexts, AS uses blinding factor r_i^t to deblind and recover the original ciphertext c_i^t of all online clients. Meanwhile, AS checks which clients' ciphertexts are missing, then records their IDs in a $DropoutID$ list and reports it to Proxy CS. Proxy CS then supplements ciphertexts for each dropout client based on the $DropoutID$ and sends them to AS.

(6) *Aggregation:* AS performs homomorphic addition to aggregate the original ciphertexts of all online clients and the supplementary ciphertexts of all dropout clients. Then, AS returns the aggregated ciphertext C^t to all online clients in the current round.

(7) *Decryption & Decoding:* Each client first decrypts the aggregated result using the decryption key s. It then decodes the obtained aggregated plaintext polynomial M^t back into the real domain, yielding the global update G^t.

Table 1
Threat model summary.

Threat type	Initiator	Method
Eavesdropping	Clients/Proxy CS/AS	Eavesdrop on and intercept messages from communication channels
Collusion attack	Clients and Proxy CS/Clients and AS	Collude to infer private data of other clients
Inference attack	Proxy CS/AS	Infer original data from received blinded plaintexts or ciphertexts

(8) Model Update: Using the global update G^t, client i updates the global model as $w^{t+1} = w^t - \eta \cdot G^t$.

Design Goals. To deliver a practical and reliable answer to the three research questions, our system must satisfy the following overarching objectives:

- *Security:* The local dataset of any client should not be leaked to other clients, AS, or Proxy CS. Basically, a single client cannot decrypt the ciphertexts of other clients. Even if less than $N-1$ clients collude with AS or Proxy CS, they are still not able to infer the private data of other honest clients. This requirement is fundamental for all three RQs.
- *Efficiency:* Clients should be able to decrypt the aggregated result locally without any interaction (addressing RQ1). Moreover, resource-constrained clients should offload heavy encryption operations to the Proxy CS, executing only lightweight operations themselves. The messages sent by clients should also be substantially smaller than full ciphertexts, further reducing communication overhead (addressing RQ2).
- *Robustness:* The system should tolerate client dropout. The training process should continue uninterrupted, and decryption should still execute correctly in the presence of dropout clients (addressing RQ3).

4.2. Design of ECMK-CKKS

To answer RQ1, we enhance xMK-CKKS [13] into ECMK-CKKS, a variant that improves the interactive decryption with single-step local decryption. Recall that in xMK-CKKS, each client encrypts using a joint public key, i.e., the sum of all clients' public keys. To decrypt any ciphertext encrypted under this key, each client must compute a decryption share using its secret key and then upload the share to the server. Only after collecting all decryption shares can the ciphertext be decrypted. While this effectively mitigates the insider threat inherent in single-key HE, the mandatory interaction among clients not only introduces extra communication rounds but also makes decryption impossible when a client drops out. ECMK-CKKS addresses the first drawback by enabling each client to decrypt locally without uploading decryption shares. The dropout issue, however, is orthogonal to the decryption mechanism and will be resolved by the secure outsourcing protocol presented in Section 4.3. In the following, we detail the construction of ECMK-CKKS.

(1) Key Generation: Given the generated public parameters PP, KDC samples for each client i a secret key sk_i from key distribution χ and computes a public key

$$pk_i = [-sk_i \cdot a + e_i]_q \ , \tag{3}$$

where $[\cdot]_q$ denotes reduction modulo q and e_i is sampled from the error distribution ψ. After generating all public and secret keys, KDC then computes the decryption key $s = \sum_{i=1}^{N} sk_i$. Finally, KDC distributes these key pairs $\{sk_i, pk_i, s\}$ to each client via secure channels. The execution of *Key Generation* is presented in Algorithm 2.

Algorithm 2 Key Generation

Input: Key distribution χ, error distribution ψ, polynomial a.
Output: Public key pk_i, secret key sk_i, decryption key s.
1: Sample a secret key sk_i from key distribution χ.
2: Sample an error e_i from error distribution ψ.
3: Compute the public key: $pk_i = [-sk_i \cdot a + e_i]_q$.
4: Compute the decryption key: $s = \sum_{i=1}^{N} sk_i$.
5: Return $\{sk_i, pk_i, s\}$.

(2) Encoding & Decoding: Before encryption, each client's real-valued message vector must be encoded into a plaintext polynomial suitable for homomorphic operations. Denote the $n/2$ messages of client i as msg_i, where n is the dimension of the polynomial. Since msg_i can be viewed as a complex vector over $C^{n/2}$ with the imaginary parts set to zero, we can map msg_i to an integral coefficient polynomial $m_i \in R$ via a scaled inverse of the canonical embedding map. Specifically, each client computes:

$$m_i = \lfloor p \cdot \pi^{-1}(msg_i) \rceil \ , \tag{4}$$

where p is the scaling factor to control the precision of encoding, π is the canonical embedding that maps a polynomial to a complex vector, and $\lfloor \cdot \rceil$ denotes rounding to the nearest integer. Decoding is simply the inverse of encoding. The procedures are summarized in Algorithm 3.

Algorithm 3 Encoding & Decoding

Input: Message msg_i, public parameters PP.
Output: Plaintext polynomial m_i.
Function 3.1: ***Ecd()***
1: Define canonical embedding π: $R[X]/(X^n+1) \rightarrow C^{n/2}$.
2: Encode: $m_i = \lfloor p \cdot \pi^{-1}(msg_i) \rceil$.
3: Return plaintext m_i.
Function 3.2: ***Dcd()***
Input: Aggregated plaintext polynomial M.
Output: Aggregated message MSG.
1: Decode: $MSG = \pi(M)/p$.
2: Return message MSG.

(3) Local Encryption: For the plaintext m_i of each client, encryption is performed using the secret key sk_i and the common polynomial $\{a, v\}$ to obtain the ciphertext

$$\begin{aligned} c_i &= (c_{i1}, c_{i2}) \\ &= \left(\left[-v \cdot a \cdot sk_i + m_i + e_{i1} \right]_q, \left[v \cdot a + e_{i2} \right]_q \right) , \end{aligned} \tag{5}$$

where e_{i1}, $e_{i2} \leftarrow \psi$. Without the knowledge of sk_i and e_{i1}, an attacker cannot infer the plaintext m_i due to the RLWE problem. The execution is shown in Algorithm 4.

Algorithm 4 Local Encryption

Input: Plaintext m_i, secret key sk_i, public parameters PP.
Output: Ciphertext c_i of client i.
1: Sample errors: $e_{i1}, e_{i2} \leftarrow \psi$.
2: Compute: $c_{i1} = [-v \cdot a \cdot sk_i + m_i + e_{i1}]_q$, $c_{i2} = [v \cdot a + e_{i2}]_q$.
3: Return ciphertext $c_i = (c_{i1}, c_{i2})$.

(4) *Aggregation:* In this process, only the first components are aggregated, and the second components serve as auxiliary material for subsequent decryption. The aggregation process is described in Algorithm 5. Then we can obtain the aggregated ciphertext C as follows:

$$\begin{aligned} C &= (C_1, C_2) \\ &= \left(\left[-v \cdot a \cdot \sum_{i=1}^{N} sk_i + \sum_{i=1}^{N} m_i + \sum_{i=1}^{N} e_{i1} \right]_q, \left[v \cdot a + e_{i2} \right]_q \right) \\ &= \left(\left[-v \cdot a \cdot s + M + e' \right], \left[v \cdot a + e_{i2} \right]_q \right), \end{aligned} \tag{6}$$

where $M = \sum_{i=1}^{N} m_i$ and $e' = \sum_{i=1}^{N} e_{i1}$. The second component $C_2 = [v \cdot a + e_{i2}]_q$ is taken from an arbitrary online client. For instance, following the procedure in Algorithm 5, it is taken from the first client's ciphertext.

Algorithm 5 Aggregation

Input: Ciphertext set $\{c_i\}_{i=1}^{N}$.
Output: Aggregated ciphertext C.
1: Set zero polynomials C_1, C_2.
2: **For** $i = 1$ to N:
3: **If** $i = 1$:
4: Add c_{i1} to C_1: $C_1 \leftarrow C_1 + c_{i1}$.
5: Add c_{i2} to C_2: $C_2 \leftarrow C_2 + c_{i2}$.
6: **Else:**
7: Add c_{i1} to C_1: $C_1 \leftarrow C_1 + c_{i1}$.
8: **End if**.
9: **End for**.
10: Return aggregated ciphertext $C = (C_1, C_2)$.

(5) *Decryption:* As detailed in Algorithm 6, each client then decrypts the aggregated ciphertext $C = (C_1, C_2)$ using the decryption key s to recover the aggregated plaintext M. The decryption process follows Algorithm 6 and is computed as:

$$\begin{aligned} M &= \left[C_1 + C_2 \cdot s \right]_q \\ &= \left[-v \cdot a \cdot s + M + e' + (v \cdot a + e_{i2}) \cdot s \right]_q \\ &= \left[-v \cdot a \cdot s + M + e' + v \cdot a \cdot s + e_{i2} \cdot s \right]_q \\ &= \left[M + e' + e_{i2} \cdot s \right]_q \\ &\approx \sum_{i=1}^{N} m_i \ . \end{aligned} \tag{7}$$

Algorithm 6 Decryption

Input: Aggregated ciphertext C, decryption key s.
Output: Aggregated plaintext M.
1: Compute: $M = [C_1 + C_2 \cdot s]_q$.
2: Return aggregated plaintext M.

Overall, compared with xMK-CKKS, ECMK-CKKS eliminates the decryption share step and enables each client to decrypt locally in one step, thereby effectively answering RQ1. However, it still fails to handle client dropout in the local encryption setting, leaving RQ3 open, which is addressed in the next subsection along with RQ2.

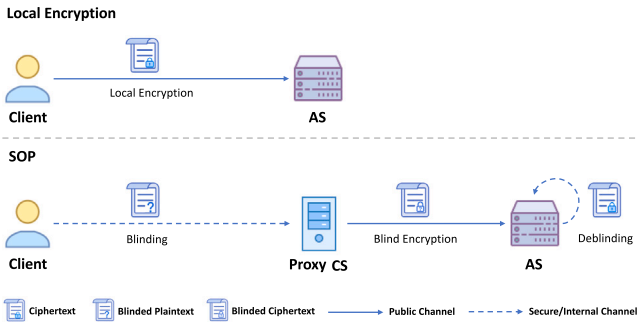


Fig. 4. Illustration of local encryption and SOP.

4.3. Secure outsourcing protocol

To jointly address RQ2 and RQ3, we design a secure outsourcing protocol SOP. As shown in Fig. 4, a client only needs to blind its plaintext and delegate homomorphic encryption to Proxy CS, which performs blind encryption and returns the resulting blinded ciphertext to AS. AS then removes the blinding term using the blinding factor to recover the original ciphertext. Algorithm 7 outlines the steps of SOP, which are detailed as follows:

(1) *Blinding:* On the client side, the plaintext m_i is locally blinded to obtain $m_i' = [m_i - r_i]_q$, where $r_i \leftarrow \mathcal{U}(R_q)$. The client then sends blinded plaintext m_i' to the Proxy CS and blinding factor r_i to the AS via secure channels, respectively.
(2) *Blind Encryption:* After receiving the blinded plaintext, Proxy CS uses the public key pk_i of each client for outsourced encryption. Specifically, Proxy CS performs the outsourced homomorphic encryption and obtains the blinded ciphertext $c_i' = (c_{i1}', c_{i2}') = \left([v \cdot pk_i + m_i' + e_{i1}]_q, [v \cdot a + e_{i2}]_q\right)$, where $e_{i1}, e_{i2} \leftarrow \psi$ and $v \leftarrow \chi$. It is worth noting that c_{i2}' does not contain blinded plaintext, so it is not blinded, and subsequent deblinding is not required. After completing the above operation, Proxy CS sends the blinded ciphertext c_i' to AS.
(3) *Deblinding:* Upon receiving c_i', since homomorphic encryption allows direct addition of ciphertext and plaintext, AS can remove the blinding term by adding the blinding factor r_i to the first component of the ciphertext, thereby recovering the original ciphertext: $c_i = \left([c_{i1}' + r_i]_q, c_{i2}'\right) = \left(\left[[v \cdot pk_i + (m_i - r_i) + e_{i1}]_q + r_i\right]_q, [v \cdot a + e_{i2}]_q\right) = \left([v \cdot pk_i + m_i + e_{i1}]_q, [v \cdot a + e_{i2}]_q\right)$.

However, the above situation assumes that all clients are online. If client dropout occurs, the SOP will enable the dropout protection mechanism to prevent decryption failure. The specific details are as follows:

(1) *Report dropout clients:* When deblinding, AS checks which clients' ciphertexts are missing, then records the IDs of these clients in $DropoutID$ and sends it to Proxy CS.
(2) *Supplement ciphertexts:* For dropout client j, Proxy CS then uses the corresponding public key to generate supplementary ciphertext $c_j = \left([v \cdot pk_j + e_{j1}]_q, [v \cdot a + e_{j2}]_q\right)$, i.e., the ciphertext for plaintext zero. After completing all dropout clients' ciphertexts, Proxy CS sends them to AS. Since the supplementary ciphertext does not contain the blinded plaintext, after receiving the supplementary ciphertext, aggregation can begin directly without removing the blinding.

SOP can seamlessly replace local encryption in ECMK-CKKS without requiring any modifications to other steps. For ease of explanation, we assume that among N clients, k clients with IDs $[1...k]$ are online,

and $N-k$ clients with IDs $[k+1...N]$ are dropped. The AS aggregates the original ciphertexts of the online clients and the supplementary ciphertexts of the dropout clients to obtain the aggregated ciphertext C as follows:

$$\begin{aligned} C &= (C_1, C_2) \\ &= \Bigg(\left[\left(v \cdot \sum_{i=1}^{k} pk_i + \sum_{i=1}^{k} m_i + \sum_{i=1}^{k} e_{i1} \right) + \left(v \cdot \sum_{j=k+1}^{N} pk_j + \sum_{j=k+1}^{N} e_{j1} \right) \right]_q , \\ &\quad \left[v \cdot a + e_{i2} \right]_q \Bigg) \\ &= \left(\left[v \cdot \sum_{i=1}^{N} pk_i + \sum_{i=1}^{k} m_i + \sum_{i=1}^{N} e_{i1} \right], \left[v \cdot a + e_{i2} \right]_q \right) \\ &= \left(\left[v \cdot \sum_{i=1}^{N} pk_i + M + e' \right], \left[v \cdot a + e_{i2} \right]_q \right), \end{aligned} \tag{8}$$

where $M = \sum_{i=1}^{k} m_i$ and $e' = \sum_{i=1}^{N} e_{i1}$. This aggregation operation is the same as the corresponding one in Section 4.2, only aggregating the first part of the ciphertext, while the second part is randomly selected from the ciphertext of one online client.

Similarly, for the decryption operation, the client can recover the aggregated plaintext using C_2 and decryption key s as follows:

$$\begin{aligned} M &= \left[C_1 + C_2 \cdot s \right] \\ &= \left[v \cdot \sum_{i=1}^{N} pk_i + M + e' + (v \cdot a + e_{i2}) \cdot s \right]_q \\ &= \left[v \cdot (-a \cdot \sum_{i=1}^{N} sk_i + \sum_{i=1}^{N} e_i \,) + M + e' + v \cdot a \cdot s + e_{i2} \cdot s \right]_q \\ &= \left[M + v \cdot \sum_{i=1}^{N} e_i + e' + e_{i2} \cdot s \right]_q \\ &\approx \sum_{i=1}^{k} m_i \ , \end{aligned} \tag{9}$$

Overall, by delegating the costly encryption operations to a proxy server, SOP operationally answers RQ2. Furthermore, by generating supplementary ciphertexts for dropout clients, it simultaneously resolves RQ3, guaranteeing correct decryption without per-round re-keying or trusted hardware.

Algorithm 7 Secure Outsourcing Protocol

Function 7.1: ***Blinding()***
Input: Plaintext m_i, public parameters PP.
Output: Blinded plaintext m_i'.
1: Sample a blinding factor: $r_i \leftarrow \mathcal{U}(R_q)$.
2: Compute blinded plaintext: $m_i' = [m_i - r_i]_q$.
3: Return blinded plaintext m_i'.

Function 7.2: ***BlindEnc()***
Input: Blinded plaintext m_i', outsourcing parameters OP, public key pk_i, fresh common polynomial $v \leftarrow \chi$.
Output: Blinded ciphertext c_i'.
1: Sample errors: $e_{i1}, e_{i2} \leftarrow \psi$.
2: Compute: $c_{i1}' = [v \cdot pk_i + m_i' + e_{i1}]_q$, $c_{i2}' = [v \cdot a + e_{i2}]_q$.
3: Return blinded ciphertext $c_i' = (c_{i1}', c_{i2}')$.

Function 7.3: ***Deblinding()***
Input: Blinded ciphertext c_i', blinding factor r_i.
Output: Original ciphertext c_i.
1: Compute: $c_{i1} = [c_{i1}' + r_i]_q$, $c_{i2} = c_{i2}'$.
2: Return original ciphertext $c_i = (c_{i1}, c_{i2})$.

Function 7.4: ***Supplement ciphertexts()***
Input: Dropout clients' IDs $DropoutID$, outsourcing parameters OP, public key set $\{pk_i\}_{i \in DropoutID}$, common polynomial v.
Output: Supplementary ciphertexts $\{c_i\}_{i \in DropoutID}$.
1: **For** i in $DropoutID$:
2: supplement $c_i = ([v \cdot pk_i + e_{i1}]_q, [v \cdot a + e_{i2}]_q)$, $e_{i1}, e_{i2} \leftarrow \psi$.
3: **End for**.
4: Return supplementary ciphertexts $\{c_i\}_{i \in DropoutID}$.

4.4. Deployment and challenges

Having described how FedOut answers the three RQs, we now discuss its practical deployment and the associated engineering challenges. To illustrate how FedOut can be deployed in a real-world FL setting with genuine privacy requirements, we consider a wearable health monitoring system. A health service provider deploys an FL application across thousands of user-owned smartwatches and fitness bands to collaboratively train an arrhythmia detection model. Each wearable device acts as an FL client, equipped with a low-power processor, limited memory, and battery constraints. These devices connect intermittently via Bluetooth or home Wi-Fi and frequently drop out when users remove them or when batteries deplete. Critically, the raw sensor data and the derived model updates contain sensitive physiological information that users are unwilling to share with other users or with the service provider, creating a strong demand for privacy protection against both external eavesdroppers and internal adversaries.

Entity Mapping. The wearable devices serve as the FL clients. The AS is operated by the health service provider in the cloud. The Proxy CS is an independent high-performance cloud server. The KDC is maintained by a separate trusted authority (e.g., a health data governance institution).

During a typical training round, each wearable device first performs on-device training using the wearer's local physiological data, then blinds the resulting updates to hide its content and transmits the blinded value to the cloud computing server. Simultaneously, the blinding factor is sent to the health provider's server. The cloud computing server encrypts the blinded data without ever seeing the original updates and forwards the encrypted result to the health provider. Upon receiving all encrypted updates, the health provider's server removes the blinding from each one, and requests the cloud computing server to generate placeholder ciphertexts for those missing devices. The server then aggregates both real and placeholder ciphertexts and distributes the aggregated result back to the active devices. Each smartwatch decrypts the aggregated update, refreshes its local arrhythmia detection model, and proceeds to the next round. The key distinction from a generic FL setting is that all resource-intensive cryptographic work is executed by the cloud computing server, while each smartwatch only performs a lightweight blinding operation that is easily accommodated within its tight energy and memory budget.

Deployment challenges. This scenario maps FedOut to a practical cross-device IoT-FL deployment, but it also reveals several challenges inherent to real-world operation.

- *Latency.* In wearable scenarios, devices connect via Bluetooth or Wi-Fi, where per-packet latency is typically modest. Moreover, FedOut mitigates the client-side latency by having clients transmit only lightweight blinded plaintexts. However, the blinded ciphertexts exchanged between the Proxy CS and the AS remain large. If these two entities are separated by a wide-area network, the transmission delay may noticeably increase the per-round latency. This can be alleviated by co-locating the Proxy CS and the AS in the same cloud region, a common practice in cloud-assisted architectures.
- *Network instability.* Wearables frequently drop out due to battery drain, user removal, or signal loss. FedOut addresses this by supplementing ciphertexts for those missing devices, which guarantees that decryption succeeds and that a training round never blocks. However, when a device is absent for multiple consecutive rounds, its local model becomes stale. Upon rejoining, it may inject outdated updates that disturb global convergence. Moreover, rounds with heavy dropout waste server-side computation and bandwidth on placeholder ciphertexts that carry no actual training data, reducing the effective throughput of the system.

- *Scalability.* Our current evaluation in Section 6.4 involves up to 50 clients, whereas a commercial deployment may encompass tens of thousands of concurrent wearables. At that scale, a single Proxy CS would be unable to sustain the blind encryption workload within an acceptable round time, and thus, encryption throughput becomes the primary bottleneck. A natural mitigation is to horizontally scale the Proxy CS into multiple instances, but this introduces additional challenges: each additional proxy instance expands the attack surface. Specifically, the original non-collusion assumption between the proxy side and the aggregation server becomes harder to enforce, as any single proxy instance colluding with the aggregation server could compromise the security of the outsourcing protocol. We leave the design of a secure and scalable multi-proxy extension for future work.

In summary, this deployment scenario concretely illustrates the engineering feasibility of our answers to the three RQs. Meanwhile, it highlights wide-area latency and scalability as open challenges that will guide future engineering efforts.

5. Security analysis

In this section, we provide a rigorous security analysis of FedOut to demonstrate that our solutions to the three RQs do not compromise security. We follow the standard ideal-world/real-world simulation paradigm [36] and prove that the protocol reveals nothing beyond the intended aggregation to any honest-but-curious adversary, provided that the underlying cryptographic primitives are secure and the two servers do not collude.

5.1. Ideal functionality and security definition

Ideal functionality. The ideal functionality for federated learning aggregation receives the local model updates m_i from each honest client and the identities of corrupted clients. It computes the sum $M = \sum_{i\in\mathcal{H}} m_i$, where $\mathcal{H}$ is the set of honest clients, and returns M to every honest client. Nothing else is leaked.

Security definition. Let Π denote the real-world FedOut protocol. For an adversary $\mathcal{A}$ that corrupts a set of parties $C \subseteq \{\text{AS}, \text{Proxy CS}, C_1, \ldots, C_N\}$, let $\text{VIEW}_{\mathcal{A}}^{\Pi}$ be the joint view of the corrupted parties during an execution of Π, including their inputs, internal randomness, and all received messages. We say that Π is secure if for every probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ corrupting C, while respecting the non-collusion constraint, there exists a PPT simulator Sim that, given only the inputs and outputs of the parties in C, according to the ideal functionality, can produce a view that is computationally indistinguishable from $\text{VIEW}_{\mathcal{A}}^{\Pi}$:

$$\text{Sim}(\text{inputs}_C, \text{outputs}_C) \overset{c}{\approx} \text{VIEW}_{\mathcal{A}}^{\Pi}. \tag{10}$$

5.2. Hardness assumptions

Our proofs rely on the following assumptions.

Definition 1 (*Ring-LWE (RLWE) [37]*)**.** Let $R_q = \mathbb{Z}_q[X]/(X^n+1)$. For a uniformly random $a \leftarrow \mathcal{U}(R_q)$, a secret $s \leftarrow \chi$, and an error $e \leftarrow \psi$, the distribution $(a, [a\cdot s + e]_q)$ is computationally indistinguishable from the uniform distribution over $R_q \times R_q$. The problem is parameterized by (n, q, χ, ψ).

In the outsourced mode, fresh $v \leftarrow \chi$ is sampled per round. For the local mode, the same v is shared among clients and its security relies on the secret distribution χ and the permanent secrecy of sk_i, as discussed in Section 5.6.

5.3. Security of basic building blocks

5.3.1. IND-CPA security of ECMK-CKKS

Lemma 1. *Let $Enc(m_i)$ be the encryption algorithm of ECMK-CKKS that produces ciphertext (c_{i1}, c_{i2}) with fresh secret $v \leftarrow \chi$. If the RLWE problem is hard, then Enc is IND-CPA secure: for any two plaintexts $m^{(0)}, m^{(1)}$ chosen by an adversary, the adversary's advantage in distinguishing their encryptions is negligible.*

Proof sketch. The ciphertext components are $c_{i2} = v\cdot a + e_{i2}$ and $c_{i1} = v\cdot pk_i + m_i + e_{i1}$. Since v is freshly sampled from χ and unknown to the adversary, the pair (a, c_{i2}) forms an RLWE sample with secret v. A standard hybrid argument (replacing c_{i2} with uniform, then c_{i1} with uniform) reduces the distinguishing advantage to twice the RLWE advantage. The full reduction is straightforward and omitted. □

5.3.2. Perfect blinding

Lemma 2. *In the SOP blinding phase, the blinded value $m'_i = [m_i - r_i]_q$ reveals no information about m_i to Proxy CS, because r_i is uniformly random over R_q and known only to the client and AS, and sent over a secure channel.*

Proof. Since $r_i \leftarrow \mathcal{U}(R_q)$ is independent of m_i and not observed by Proxy CS, the distribution of m'_i is uniform over R_q regardless of m_i. Hence, the view of Proxy CS is statistically independent of m_i. □

5.4. Security of the Secure Outsourcing Protocol (SOP)

We now prove that the SOP component securely realizes the encryption outsourcing and dropout recovery functionality against semi-honest adversaries who respect the non-collusion assumption.

Theorem 1. *Under the RLWE assumption and with secure channels for client–Proxy CS and client–AS communication, the SOP protocol securely computes the outsourced encryption and ciphertext recovery in the presence of semi-honest adversaries, provided that AS and Proxy CS do not collude.*

Proof by simulator construction. We examine the possible sets of corrupted parties.

Case 1: Corrupted AS. The real view of AS consists of the blinding factors r_i from online clients, the blinded ciphertexts c'_i from Proxy CS, and, for dropout clients, the supplementary ciphertexts c_j from Proxy CS. We construct a simulator Sim_{AS} that works as follows:

1. Sim_{AS} chooses random $r_i^* \leftarrow \mathcal{U}(R_q)$ for each online client.
2. It generates random ciphertexts $c_i'^* \leftarrow \mathcal{U}(R_q \times R_q)$ for the first component of blinded ciphertexts, and similarly for the second component.
3. For dropout clients, it generates random supplementary ciphertexts $c_j^* \leftarrow \mathcal{U}(R_q \times R_q)$.

The random blinding factors are perfectly indistinguishable from the real r_i. By Lemma 1, the real blinded ciphertext c'_i, which is an ECMK-CKKS encryption of m'_i, is computationally indistinguishable from a uniform random polynomial pair. The supplementary ciphertexts are encryptions of zero with fresh v, so they are equally indistinguishable from random. Thus, the simulated view is computationally indistinguishable from the real view.

Case 2: Corrupted proxy CS. The view of Proxy CS includes the blinded plaintexts m'_i received from each online client and the outsourcing parameters OP. Moreover, υ is sampled locally by Proxy CS. We construct $\mathsf{Sim}_{\mathsf{Proxy\ CS}}$ as follows:

1. $\mathsf{Sim}_{\mathsf{Proxy\ CS}}$ generates dummy blinded messages $m'^{*}_i \leftarrow \mathcal{U}(R_q)$ for each client, simulating the real distribution, which is uniform by Lemma 2.
2. It outputs the tuple $(OP, \{m'^{*}_i\})$.

The only difference from the real execution is that the real m'_i are computed as $[m_i - r_i]_q$, whereas the simulated ones are purely random. However, by Lemma 2, the distribution of m'_i is exactly uniform over R_q, independent of m_i. Hence, the real and simulated views are identically distributed.

Case 3: Corrupted client C_i. A semi-honest client knows its own local update m_i, its key pair (sk_i, pk_i), the decryption key s, the aggregated ciphertext C received from AS, as well as the global model update in plaintext after decryption. The client does not observe any other client's blinded plaintext or blinding factor because they are transmitted over secure channels. The simulator can trivially reproduce this view by running the protocol honestly with the same inputs and outputs. Therefore, the client learns nothing about other clients' updates beyond the aggregated sum, which it is allowed to learn.

Case 4: Collusion between AS and less than $N-1$ clients. If AS colludes with less than $N-1$ corrupted clients, the adversary additionally obtains the secret keys and local updates of those clients. The simulator for this coalition can run $\mathsf{Sim}_{\mathsf{AS}}$ as before, but also embed the known inputs/outputs of the corrupted clients. The indistinguishable property still holds because the random ciphertexts for honest clients remain indistinguishable from real ciphertexts by Lemma 1 and the blinding factors for honest clients are independent. Moreover, because the decryption key s is the sum of all sk_i, the adversary can decrypt the aggregated ciphertext to obtain M. Subtracting its known updates yields the sum of the honest clients' updates, but nothing finer. This is exactly the information permitted by the ideal functionality.

Case 5: Collusion between proxy CS and less than $N-1$ clients. In this case, Proxy CS already holds the blinded messages of honest clients, which are uniform and thus give no information. Any additional knowledge of corrupted clients' data does not help to break the uniformity. The simulator can again produce a view with uniformly random blinded plaintexts for honest clients, maintaining indistinguishability.

Case 6: Collusion between AS and proxy CS. This case is explicitly prohibited in our threat model. If it occurred, the two servers would together possess both the blinding factors r_i and the blinded plaintexts m'_i, allowing them to recover all individual m_i. Since we assume non-collusion, this attack is out of scope.

Thus, in all admissible corruption scenarios, we have constructed simulators satisfying the definition of security. This completes the proof of Theorem 1. ■

5.5. Security of the full FedOut framework

Theorem 2. *Under the same assumptions as Theorem 1 and with the additional trust in the KDC, the entire FedOut protocol securely realizes the ideal FL aggregation functionality in the semi-honest model.*

Proof sketch. The full FedOut execution consists of repeating the SOP phase in each training round, interleaved with local model training on genuine inputs. The local training phase is entirely local to each client and does not involve communication. Thus, it reveals nothing. The aggregation phase is precisely an instantiation of SOP, whose security we have proved in Theorem 1. The decryption phase uses the common decryption key s to recover the aggregated plaintext, which is then used to update the global model. Because the aggregation result is the only new information revealed at the end of each round, and it is exactly the sum of the current round's updates, the overall view of any PPT adversary can be simulated round-by-round by invoking the SOP simulators and feeding them the appropriate round-wise sums provided by the ideal functionality. Simultaneously, the KDC is trusted and only distributes initial keys, and its operations are trivially simulatable. Consequently, the multi-round FedOut protocol remains secure. □

5.6. Discussions on advanced security properties

Multi-round privacy and dropout resilience. In FedOut, the decryption key $s = \sum_{i=1}^{N} sk_i$ is fixed at setup and never changes. When a client drops out, the Proxy CS generates a zero-valued supplementary ciphertext under that client's public key. Hence, the aggregation sum always includes a zero contribution from the dropout, and the same static s can decrypt correctly. Because s never varies, an adversary cannot exploit differing decryption keys across rounds to extract individual secret keys, as would be possible in schemes that recompute s from the online set. Furthermore, in the outsourced mode, every ciphertext is encrypted with a freshly sampled υ, making cross-round ciphertexts unlinkable. Together, these mechanisms provide strong multi-round privacy under the honest-but-curious model.

Post-quantum security. The core encryption ECMK-CKKS relies exclusively on the RLWE problem, which is believed to be hard even for quantum computers. The blinding method in SOP is information-theoretically secure, i.e., a one-time pad, and does not involve integer factorization or similar classically hard assumptions. Therefore, the entire FedOut framework is *post-quantum secure* under the RLWE assumption. No component of the protocol depends on discrete-log or factoring problems.

Justification of the non-collusion assumption. The assumption that AS and Proxy CS do not collude is standard in two-server privacy-preserving computation [34,35]. In practice, these two servers can be operated by independent cloud providers or isolated administrative domains [35], with contractual obligations and periodic audits that make collusion economically and legally prohibitive. For environments where this assumption is too strong, one can employ trusted execution environments (TEEs) for the Proxy CS or introduce zero-knowledge proofs to detect misbehavior. We leave these extensions for future work.

Local mode security remark. In the local encryption mode, the polynomial υ is a fixed secret parameter distributed by the KDC to all clients. Indistinguishability of ciphertexts under this mode still holds under the RLWE assumption because the adversary, who may corrupt the AS or other clients, does not know an honest client's sk_i.

6. Performance evaluation

In this section, we experimentally validate whether FedOut satisfies the three research questions. Specifically, we assess the decryption precision of ECMK-CKKS, the model accuracy under client dropout, and the computational and communication gains achieved by offloading encryption.

6.1. Experimental setup

Platform and parameters: In our experimental setup, a Raspberry Pi 4B was used to simulate resource-constrained clients, while a high-performance server equipped with an Intel Xeon Platinum 8470Q CPU (25 vCPUs) served as both the outsourcing and aggregation server. The multi-key algorithm was implemented by extending the HEAAN

Table 2
Comparison of decryption errors for local and outsourced mode.

	Mean	Max	P99	SD
Local	10^{-9}	10^{-7}	10^{-9}	10^{-9}
Outsourced	10^{-9}	10^{-7}	10^{-9}	10^{-9}

library [38]. Specifically, we developed ECMK-CKKS in C++, utilizing the NTL 11.6.0 [39] and GMP 6.3.0 [40] libraries for arbitrary-length integer arithmetic and arbitrary-precision computations. Federated learning training was conducted in a PyTorch 2.7.0 and Python 3.12 environment. To integrate the homomorphic encryption algorithm into the federated learning pipeline, we compiled it into a dynamic library that could be called from Python.

We set the degree of the ring $\log n = 15$ to match ESAFL [17] for fair comparison, and the scaling factor $\log p = 50$ to ensure adequate precision, as validated by our precision experiments in Section 6.2. The key distribution χ follows a uniform ternary distribution over $\{-1, 0, 1\}$, and the error distribution ψ is a discrete Gaussian with standard deviation $\sigma = 3.2$, as recommended in security guidelines for implementing homomorphic encryption [41]. We then evaluate the security of this configuration using the Lattice Estimator [42], a commonly used tool for estimating the concrete security of FHE parameter sets. The evaluation shows that the maximum ciphertext modulus satisfying the 256-bit security level is $\log q = 462$, which agrees with the result in [41]. Hence, we set $\log q = 462$, as it guarantees the targeted security while maximizing the homomorphic capacity.

Datasets and models: We evaluated our proposed scheme on two real datasets: FMNIST and CIFAR-10. For FMNIST, a 3-layer fully-connected neural network with 101.77 K weights was employed for training, using a batch size of 128 and the Adam optimizer. For CIFAR-10, we trained the model using AlexNet with 1.25M weights, with a batch size of 128 and the RMSprop optimizer with 10^{-6} decay. In both experiments, the dataset of each client followed an independent and identically distributed (IID) partition.

Baselines: To validate our proposed scheme, we introduce the following baselines for comparison.

- *Plaintext:* This approach employs direct plaintext transmission and aggregation, utilizing no privacy-preserving mechanisms, and thus incurs the lowest computational and communication overhead.
- *BatchCrypt:* single-key HE FL [43], which leverages quantization and batch encoding of gradients to minimize homomorphic encryption operations and data transfer.
- *ESAFL:* multi-key HE FL [17], which leverages a polynomial packing method and fast Fourier transform (FFT) for fast polynomial multiplication in plaintext encryption.
- *FedPHE:* single-key HE FL [11], which mitigates computation and communication overheads through pack-level sparsification and sketch-based straggler-resistant client selection.
- *VeriFL:* SA-based FL [4], which tolerates client dropout through secret sharing and a double-masking mechanism. The server collects sufficient secret shares from online clients to reconstruct the masks, thereby removing the masks of the dropout clients.
- *SecFed:* TEE-based FL [20], in which the dropout protection mechanism leverages a trusted TEE to generate supplementary ciphertexts for dropout clients.

6.2. Decryption precision

To evaluate the decryption precision of ECMK-CKKS, we used 16384 test samples and measured the error by computing the difference between the decrypted data and the original messages. Table 2 presents the statistical results of decryption errors under both local encryption and outsourced encryption. As shown, ECMK-CKKS achieves excellent decryption precision: the P99 error is on the order of 10^{-9}, meaning that 99% of the decryption errors are below 10^{-9}. The standard deviation (SD) and the mean are also on the order of 10^{-9}, indicating that the decryption precision remains stable. The maximum observed error is only on the order of 10^{-7}. Moreover, outsourcing introduces no additional loss of precision: the decryption precision under outsourced encryption is identical to that of local encryption. In summary, the enhanced homomorphic encryption algorithm delivers outstanding decryption precision in both local and outsourced encryption modes.

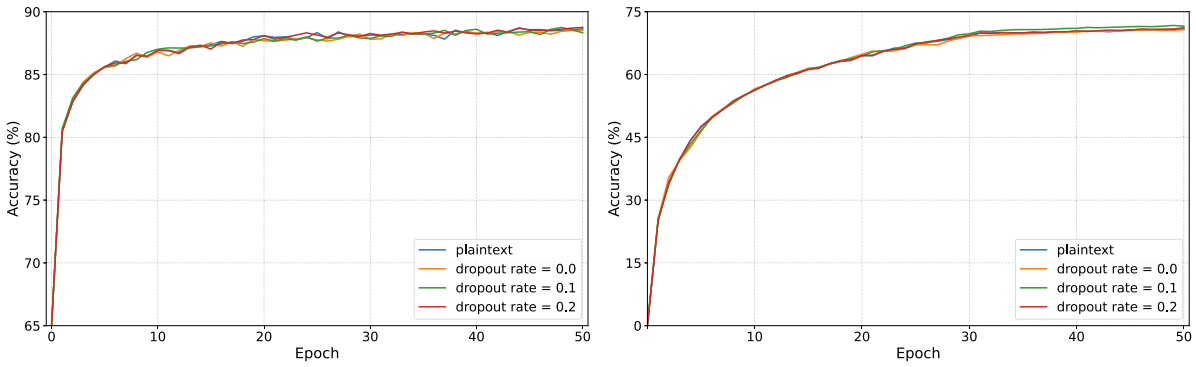
6.3. Model accuracy

To evaluate the impact of client dropout on model accuracy, we compared the performance of FedOut under different dropout rates with the plaintext baseline, as shown in Fig. 5. For the FMNIST dataset, the plaintext baseline reached its highest accuracy of 88.76% at round 48, while FedOut without dropout achieved 88.75% at round 50, demonstrating that the encryption and outsourcing mechanisms do not degrade model quality. When the dropout rate was set to 0.1 and 0.2, the best accuracies of FedOut were 88.75% at round 48 and 88.52% at round 50, respectively, reflecting only a slight decrease under heavier dropout conditions. For CIFAR-10, due to the long training time, we limited the comparison to the first 50 rounds: plaintext attained 70.87% at round 50, while FedOut without dropout reached 71.18%. With dropout rates of 0.1 and 0.2, the accuracies were 71.56% and 70.69%, respectively. Notably, the accuracy at a 0.1 dropout rate is slightly higher than that without dropout. This can be attributed to the regularization effect of client dropout: randomly excluding a subset of clients in each round prevents the global model from overfitting to the local data distributions of individual clients, thereby improving generalization, a phenomenon also observed in prior federated learning studies [44,45]. Overall, FedOut maintains model accuracy on par with or even slightly above the plaintext baseline across different participation conditions, and the supplementary-ciphertext dropout protection effectively preserves convergence stability even when a fraction of clients disconnect. These results confirm that the proposed framework achieves both strong privacy guarantees and robust training performance in realistic FL settings.

6.4. Computational cost

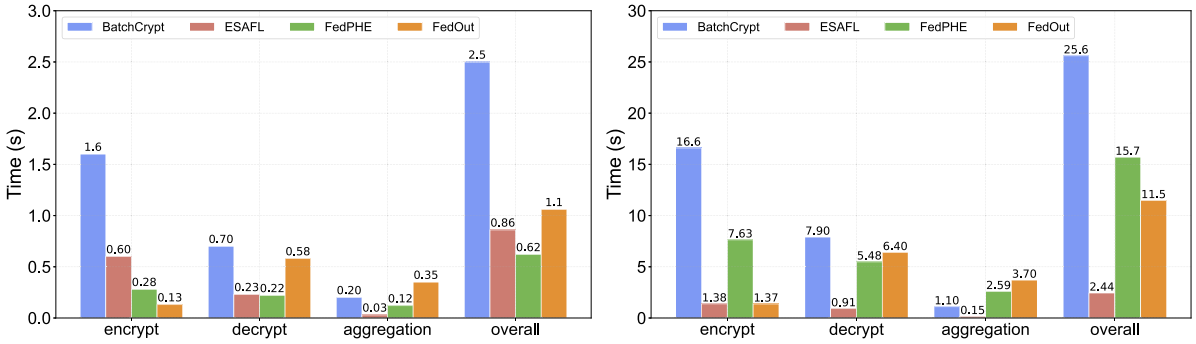
FedOut versus Baselines: Fig. 6 reports the per-round computation time of FedOut, BatchCrypt, ESAFL, and FedPHE on the two datasets. In BatchCrypt, ESAFL, and FedPHE, every client must execute a full homomorphic encryption on its local update, while FedOut replaces this heavy operation with a lightweight blinding procedure. On FMNIST, the client-side encryption cost of FedOut is only 0.13 s, compared to 1.6 s for BatchCrypt, 0.6 s for ESAFL, and 0.28 s for FedPHE. On CIFAR-10, FedOut spends 1.37 s, whereas the other three schemes require 16.6 s, 1.38 s and 7.63 s, respectively. Since BatchCrypt, ESAFL, and FedPHE use batching or sparsification to reduce the number of ciphertexts, the aggregation and decryption stages of FedOut process a larger volume of ciphertexts, resulting in higher costs in these two steps, yet the overall per-round latency stays competitive. On FMNIST, FedOut completes one round in 1.06 s, which is slightly more than ESAFL and FedPHE but 2.4× faster than BatchCrypt. On CIFAR-10, FedOut requires 11.47 s per round, outperforming BatchCrypt with a 2.2× speedup, and FedPHE with a 1.4× speedup, while being slower than the highly-packed scheme ESAFL. These results confirm that the outsourcing design of FedOut effectively removes the most computationally demanding operation from resource-constrained clients, and the overall training efficiency improves significantly over BatchCrypt and FedPHE, with the gap widening as the model size grows.

Computational gains of SOP over local encryption: To isolate the source of computational savings, we compare local ECMK-CKKS encryption against three key configurations: the client-side blinding step,



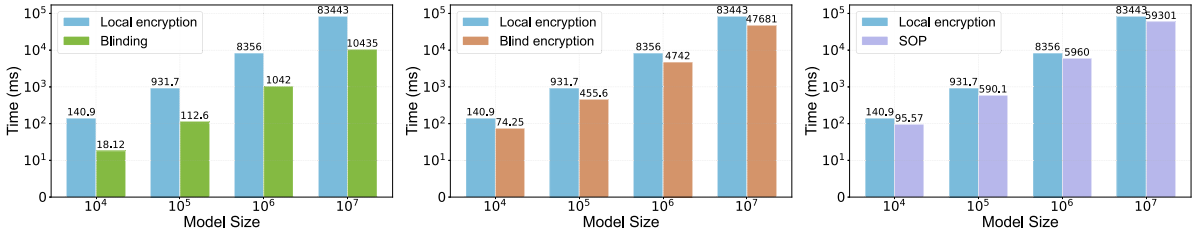
(a) FMNIST (b) CIFAR-10

Fig. 5. Accuracy of plaintext baseline and FedOut under different dropout rates.



(a) FMNIST (b) CIFAR-10

Fig. 6. Comparison of computational cost between FedOut and baselines under different datasets.



(a) Local encryption vs. Blinding (b) Local encryption vs. Blind encryption (c) Local encryption vs. SOP

Fig. 7. Component-wise breakdown of computational cost: client blinding, proxy blind encryption, and full SOP.

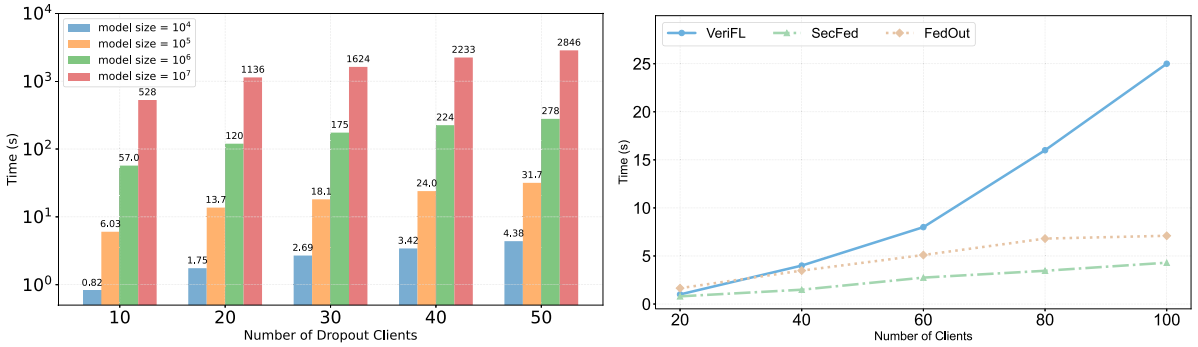
Table 3
Comparison of peak memory occupancy (MB) for local encryption vs. blinding under different $\log n$.

	$\log n = 12$	$\log n = 13$	$\log n = 14$	$\log n = 15$
Loc_enc	6.33	11.16	20.75	39.95
Blinding	2.49	3.83	6.42	11.63

the proxy-side blind encryption step, and the complete SOP pipeline. The measurements for model sizes ranging from 10^4 to 10^7 are shown in Fig. 7. At the largest model, local encryption on the Raspberry Pi consumes 83.44 s, whereas the client-side blinding operation takes only 10.44 s, reaching an 8× reduction, confirming that the client is almost completely freed from heavy cryptographic work. The blind encryption executed by the high-performance Proxy CS requires 47.68 s, which is approximately 1.75× faster than what the resource-constrained client would need for the same operation, verifying the proxy's hardware advantage. When all three steps are combined, the end-to-end SOP latency is 59.30 s, still 1.4× lower than local encryption while entirely offloading the encryption workload from the client. The dominant component is blind encryption, whose time scales roughly linearly with model size. However, this cost is borne by the scalable proxy, not by the IoT device. Overall, SOP effectively decomposes encryption, retains only a lightweight modulo subtraction on the client, and redirects the computationally intensive polynomial arithmetic to a powerful server.

We further measured the peak memory of client-side local encryption and the blinding step under different ring dimensions $\log n$. As shown in Table 3, memory usage of both operations grows with n, but blinding is significantly lighter: at $\log n = 15$, local encryption consumes 39.95 MB while blinding uses only 11.63 MB, reaching a 3.4× reduction. The large gap stems from the nature of the two operations: local encryption involves expensive polynomial modular multiplication, plus key storage, ciphertext components, and error sampling, all of which require substantial temporary buffers, whereas blinding performs only a single modular subtraction. Hence, the outsourcing design makes FedOut safe for memory-constrained devices even under strong security parameters.

Time breakdown of SOP: Table 4 details the per-stage latency of SOP for varying client counts. The bottleneck lies clearly in the blind encryption step on the Proxy CS, whose time grows roughly linearly with



(a) Comparison of recovery time across different model sizes.

(b) Comparison of recovery time across different schemes.

Fig. 8. Dropout recovery overhead.

Table 4
Breakdown of computational cost in SOP under different numbers of clients (model size = 10^6).

Number of clients	Blinding (s)	Blind encryption (s)	Deblinding (s)	Overall (s)
10	1.05	49.50	1.68	52.23
20	1.05	103.59	3.41	108.05
30	1.05	156.18	4.89	162.12
40	1.05	199.54	7.06	207.65
50	1.05	267.73	8.45	277.23

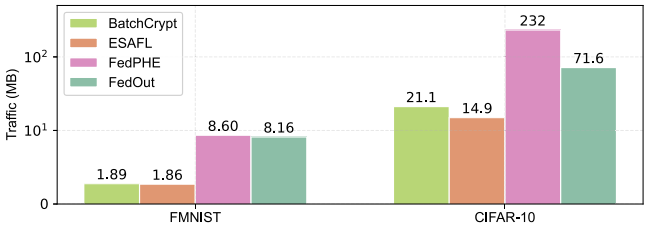
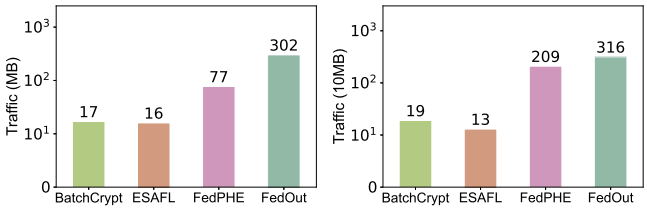


Fig. 9. Comparison of communication overhead on client-side.



(a) FMNIST **(b)** CIFAR-10

Fig. 10. Comparison of per-round communication overhead among schemes.

the number of clients and dominates over 94% of the total SOP cost. In contrast, the client-side blinding time remains constant regardless of the client count, because blinding is executed independently and in parallel on each device. Deblinding on the AS contributes only a marginal fraction of the overall latency. These results confirm that the efficiency of SOP is primarily determined by the computing capability of the Proxy CS, which, as a pluggable module, can be upgraded or scaled out to further reduce blind encryption time.

Dropout recovery overhead: When clients dropout occurs, the AS reports the identifiers of missing clients to the Proxy CS, which generates a supplementary zero-plaintext ciphertext for each dropout client using its public key. This operation is a standard ECMK-CKKS encryption whose per-ciphertext cost equals that of blind encryption. The total recovery time scales linearly with both the model size and the number of dropped clients. For a moderate model of 10^6 parameters and 50 dropout clients, it reaches only 278.3 s, which is a small fraction of a typical training round. A comparison with VeriFL and SecFed under a 40% dropout rate is provided in Fig. 8(b). Both SecFed and FedOut handle dropout by generating supplementary ciphertexts. However, SecFed relies on trusted hardware for this generation, introducing hardware dependencies, whereas VeriFL requires extra communication rounds with online clients to cancel masks, resulting in substantial latency. FedOut avoids both client communication and specialized hardware by generating supplementary ciphertexts directly on the Proxy CS, keeping recovery entirely computation-bound. The results show that FedOut's recovery time is substantially lower than that of VeriFL and comparable to that of SecFed, while remaining purely software-based and transparent to resource-constrained clients.

6.5. Communication cost

Communication cost versus baselines: Fig. 9 reports the per-round communication cost of FedOut, BatchCrypt, ESAFL, and FedPHE on the client-side. BatchCrypt and ESAFL employ batching techniques that pack multiple plaintexts into a single ciphertext, yielding a low client upload of only 1.89 MB and 1.86 MB on FMNIST, and 21.11 MB and 14.94 MB on CIFAR-10. FedPHE reduces client-side traffic through pack-level sparsification, sending 8.60 MB with 20% sparsification ratio on FMNIST and 232.2 MB with 70% sparsification ratio on CIFAR-10. FedOut replaces ciphertexts with lighter blinded plaintexts and blinding factors, resulting in client uploads of 8.16 MB on FMNIST and 71.60 MB on CIFAR-10, which are comparable to FedPHE on the smaller dataset and about 3.2× lower on the larger one. However, the outsourcing protocol additionally transmits full-size blinded ciphertexts between the proxy and the aggregation server. Thus, as shown in Fig. 10, the total system traffic of FedOut reaches 301.86 MB on FMNIST and 315.65 MB on CIFAR-10, which is higher than that of the other schemes. Overall, FedOut offloads the heavy ciphertext transmission from the resource-constrained clients to the high-bandwidth proxy–aggregator link, keeping the client-side cost competitive with packing-based and sparsification-based approaches at the expense of increased total network usage.

Traffic breakdown of FedOut: Fig. 11 details the sizes of the different messages involved in one round of FedOut. On FMNIST, the blinded plaintext and the blinding factor sent by each client together amount to 8.15 MB, which represents a 3.1× reduction compared to the 25.38 MB ciphertext that would be transmitted under local encryption. On CIFAR-10, the upload per client totals 71.59 MB, yielding a 3.9× reduction over the 279.13 MB ciphertext. Notably, the blinding factor size remains constant regardless of the dataset, as it is a single random polynomial determined solely by the ring parameters, whereas both the blinded plaintext and the ciphertext scale with the model size. The largest communication component remains the blinded ciphertext

Table 5
Ablation study for ECMK-CKKS.

Dataset	Enc. Algo.	Agg. time (ms)	Dec. time (ms)	Dec. Mem. (MB)	Dec. Traff. (MB)	Dec. engagement
FMNIST	xMK-CKKS	709.71	713.52	38.72	1.81	✓
	ECMK-CKKS	349.60	575.12	38.63	0	✗
CIFAR-10	xMK-CKKS	7669.29	7438.85	38.72	1.81	✓
	ECMK-CKKS	3755.08	6399.21	38.63	0	✗

Note: We abbreviate Encryption as Enc., Algorithm as Algo., Aggregation as Agg., Decryption as Dec., Memory as Mem., Traffic as Traff.

Table 6
Ablation study for SOP.

Dataset	SOP	Client-side Comp. (ms)	Client-side Mem. (MB)	Client-side Traff. (MB)	Per-round HE latency (ms)	Dropout tolerance
FMNIST	✗	1852.39	39.95	25.38	2211.56	✗
	✓	1078.18	11.63	8.15	6672.13	✓
CIFAR-10	✗	20 485.03	39.95	279.13	24 302.33	✗
	✓	11 836.32	11.63	71.59	74 215.37	✓

Note: We abbreviate Computational Cost as Comp., Memory as Mem., Traffic as Traff.

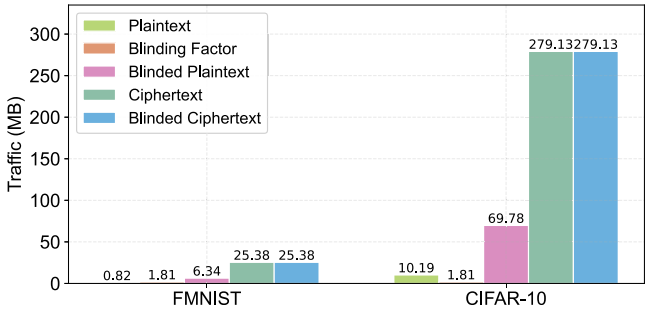


Fig. 11. Comparison of different message types.

Table 7
Comprehensive comparison of accuracy, training time and network traffic under different training modes.

Dataset	Mode	Epochs	Accuracy	Time (h)	Traffic (GB)
FMNIST	BatchCrypt	68	88.37%	8.9	58.7
	ESAFL	34	88.79%	0.6	28.9
	FedPHE	42	88.64%	0.4	165.8
	FedOut	50	88.75%	1.3	766.4
	Plaintext	48	88.76%	0.2	17.2
CIFAR-10	BatchCrypt	279	74.04%	131.3	227.8
	ESAFL	255	73.85%	10.2	180.6
	FedPHE	224	73.61%	28.5	2234.9
	FedOut	248	73.47%	45.7	3737.4
	Plaintext	226	73.89%	3.4	98.4

exchanged between the Proxy CS and the AS, whose size is identical to that of the original ciphertext. These results confirm that SOP effectively shifts the heavy ciphertext transmission away from the resource-constrained clients: clients send only the combined blinded plaintext and blinding factor, while the full-size ciphertext payloads are confined to the high-bandwidth proxy–aggregator link.

6.6. Ablation study

To separately quantify the contributions of the enhanced multi-key scheme and the outsourcing protocol, we conduct two ablation studies. Table 5 compares ECMK-CKKS with its predecessor xMK-CKKS. ECMK-CKKS reduces the aggregation time by roughly 50% on both datasets and likewise shortens the decryption time. More importantly, it completely eliminates the client-side communication overhead for decryption and removes the need for any client engagement in the decryption process. Table 6 isolates the effect of SOP by comparing the local encryption mode with the outsourced mode. Activating SOP significantly reduces client-side computation, memory, and traffic: on CIFAR-10, client computation drops from 20 485 ms to 11 836 ms, memory from 39.95 MB to 11.63 MB, and upload traffic from 279.13 MB to 71.59 MB. The per-round HE latency increases under SOP because, in the local mode, encryption is distributed across all clients and executed in parallel, whereas the outsourced mode concentrates the entire encryption workload onto a single Proxy CS that processes the blinded plaintexts sequentially. This serialization bottleneck highlights the scalability challenge of the proxy, as discussed in Section 4.4, and can be alleviated by horizontally scaling the Proxy CS into multiple instances in practice. Critically, SOP equips the system with dropout tolerance, an indispensable feature for real-world IoT deployments that the local encryption mode lacks. Taken together, the ablation results confirm that ECMK-CKKS tightens security and simplifies decryption with minimal overhead, while SOP shifts the heavy cryptographic burden from resource-constrained clients to the proxy, simultaneously adding robustness against client churn.

6.7. Comprehensive comparison

Finally, we trained all schemes to convergence and summarized the results in Table 7. All methods achieve accuracy close to plaintext. On FMNIST, FedOut completes in 1.3 h, outperforming BatchCrypt but slower than ESAFL and FedPHE. On CIFAR-10, FedOut requires 45.7 h, again beating BatchCrypt yet trailing ESAFL and FedPHE. This gap arises because FedOut does not employ packing or sparsification, resulting in a larger ciphertext volume that increases blind encryption, aggregation, and decryption time. Similarly, the total communication overhead of FedOut is significantly higher than that of the baselines, yet its client-side upload remains competitive and all encryption work is offloaded to the proxy. Overall, FedOut drastically reduces client-side computation and transmission at the cost of increased total network usage, which motivates future integration of packing or sparsification into the outsourcing protocol.

7. Discussion

Having presented the design of FedOut, its formal security analysis, and its empirical evaluation, we now synthesize this evidence to explicitly delineate how each research question is operationally answered: from design rationale, through security guarantees, to experimental validation.

7.1. Answering RQ1: Non-interactive decryption

Design rationale. RQ1 asks how multi-key HE can be redesigned to support single-step local decryption without any client interaction. FedOut answers this question through ECMK-CKKS in Section 4.2. Each client encrypts its local updates using its own secret key sk_i together with shared polynomials a and v, thereby embedding a client-specific term that depends on sk_i into the ciphertext. Instead of each client computing and uploading a decryption share, all secret keys sk_i are aggregated into a single decryption key $s = \sum_{i=1}^{N} sk_i$ at setup time. As shown in Eq. (7), the aggregated ciphertext $C = (C_1, C_2)$ then contains a combined interference term $(-v \cdot a \cdot s)$ that can be canceled locally by any client using the pre-distributed key s: $M = [C_1 + C_2 \cdot s]_q$. Thus, decryption is completed in a single local step, without any communication with other clients or the server.

Security assurance. A natural concern is whether adapting the homomorphic encryption algorithm to enable single-step decryption compromises privacy. Lemma 1 in Section 5 establishes that ECMK-CKKS is IND-CPA secure under the RLWE assumption. Since the decryption key s is the sum of all secret keys, a single client cannot recover any individual sk_i from s, and the ciphertext components (C_1, C_2) remain computationally indistinguishable from uniform random elements. This guarantees that the non-interactive decryption step does not create any additional leakage compared to the original interactive protocol.

Experimental validation. Table 2 demonstrates that the decryption precision of ECMK-CKKS reaches 10^{-9} (P99 error), confirming that the single-step decryption introduces no meaningful accuracy loss. The ablation study in Table 5 further shows that ECMK-CKKS reduces the aggregation time by approximately 50% and completely eliminates the client-side communication overhead for decryption shares, compared with xMK-CKKS.

7.2. Answering RQ2: Secure offloading

Design rationale. RQ2 asks how the RLWE-based HE encryption workload can be securely offloaded to a proxy server, so that clients only perform lightweight operations. FedOut addresses this through SOP in Section 4.3. A client blinds its plaintext with a random factor r_i before sending it to the Proxy CS. The proxy then performs the expensive encryption on the blinded value and forwards the blinded ciphertext to the AS. The AS finally removes the blinding to recover the original ciphertext. Critically, the blinding operation is a single modular subtraction, which is fundamentally lighter than the large-degree polynomial multiplication required by RLWE encryption. Moreover, the blinded plaintext transmitted by the client is significantly smaller than a full ciphertext.

Security assurance. The security of SOP is established by Theorem 1 in Section 5, which proves that the protocol reveals nothing beyond the intended aggregation to any honest-but-curious adversary, provided that the AS and the Proxy CS do not collude. Lemma 2 further guarantees that the blinded value $m_i' = [m_i - r_i]_q$ is uniformly distributed over R_q and thus statistically independent of the actual plaintext m_i, ensuring perfect hiding from the proxy. Together, these results prove that outsourcing does not weaken the privacy guarantees of the underlying HE scheme.

Experimental validation. The experimental evidence directly supports the efficiency claim of RQ2. On the computational side, Fig. 6 shows that the client-side encryption cost on CIFAR-10 drops by over 12× compared with BatchCrypt. Fig. 7 further decomposes this gain: at model size 10^7, local encryption takes 83.44 s, whereas blinding takes only 10.44 s. Table 4 confirms that the blind encryption cost is shifted to the high-performance Proxy CS, and Table 3 demonstrates that the peak memory footprint of blinding is far lower than that of local encryption. On the communication side, Fig. 9 shows that FedOut achieves notably lower client-side traffic than FedPHE on CIFAR-10, while remaining competitive with the batching-based schemes on both datasets. Fig. 11 details that the combined size of the blinded plaintext and the blinding factor is significantly smaller than a full ciphertext. Together, these results confirm that both the encryption computation and the associated ciphertext transmission are successfully offloaded from resource-constrained clients to the proxy.

7.3. Answering RQ3: Dropout resilience

Design rationale. RQ3 asks how the FL framework can maintain correct decryption and stable model convergence when clients drop out, without interrupting training. FedOut answers this through the supplementary-ciphertext mechanism in SOP in Section 4.3. When a client drops out, the AS reports the dropout ID list to the Proxy CS, which generates a zero-plaintext ciphertext under that client's public key. These supplementary ciphertexts provide the missing ciphertext components that would have been contributed by the dropout clients had they remained online. As a result, the aggregated ciphertext retains the same algebraic structure as in the full-participation case, and the static decryption key $s = \sum sk_i$ can decrypt it correctly without any per-round re-keying, as shown in Eq. (9).

Security assurance. The dropout recovery mechanism is secure for a simple reason: the supplementary ciphertext is an encryption of zero, and a zero-plaintext ciphertext contains no private information about the dropout client. Generating this supplementary ciphertext is a special case of the blind encryption performed in SOP: the Proxy CS encrypts a blinded value under the client's public key, with the blinded value being zero. Since Theorem 1 already establishes that the entire SOP protocol reveals nothing beyond the intended aggregation, this special case inherits the same security guarantee. Therefore, the dropout recovery mechanism operates without weakening the overall security model.

Experimental validation. Fig. 5 demonstrates that FedOut maintains model accuracy within 0.5% of the plaintext baseline even under a 20% dropout rate on both FMNIST and CIFAR-10. Fig. 8 further shows that the dropout recovery overhead of FedOut is substantially lower than that of the communication-heavy VeriFL and comparable to the TEE-based SecFed, while remaining purely software-based and transparent to clients.

7.4. Summary

The above analysis demonstrates that FedOut systematically answers all three research questions within a unified framework. ECMK-CKKS enables non-interactive local decryption with provable IND-CPA security (RQ1). SOP securely offloads RLWE-based encryption to a proxy server, leaving clients with only lightweight blinding operations (RQ2). The supplementary-ciphertext mechanism guarantees dropout resilience without re-keying or trusted hardware (RQ3). Together, these contributions, validated through formal security proofs and extensive experiments on real-world datasets, establish FedOut as a secure, efficient, and robust FL framework tailored for resource-constrained IoT devices.

8. Conclusion

In this paper, we presented FedOut, a secure, efficient, and robust FL framework designed to address three critical research questions in multi-key HE-based federated learning. These goals are achieved through the integration of ECMK-CKKS, an enhanced multi-key CKKS scheme that enables non-interactive single-step local decryption, and SOP, a secure outsourcing protocol that offloads the heavy RLWE encryption workload from resource-constrained clients to a proxy server while providing dropout resilience via supplementary ciphertexts. We formally proved that these designs do not compromise security under

the honest-but-curious model with the two-server non-collusion assumption, and extensive experiments on FMNIST and CIFAR-10 demonstrated that FedOut reduces client-side computational latency and communication overhead compared to HE-based SOTA baselines, while maintaining plaintext-level accuracy. We further synthesized the evidence from the design, security analysis, and experimental evaluation into a comprehensive discussion of how each research question is operationally answered.

However, the current design relies on a single Proxy CS that sequentially encrypts all clients' blinded plaintexts, creating a performance bottleneck as the number of clients increases. Future work will explore multi-proxy architectures that distribute proxy requests across multiple proxy instances to enhance scalability and further reduce training time. Moreover, integrating packing or sparsification techniques into the outsourcing pipeline remains a promising direction to compress the overall network traffic introduced by ciphertext expansion.

CRediT authorship contribution statement

Yi Yuan: Writing – original draft. **Hong Rao:** Conceptualization. **Shuanggen Liu:** Supervision. **Rixuan Qiu:** Project administration. **Mengyuan Cui:** Data curation. **Yuxin Hu:** Investigation.

Declaration of competing interest

a. This manuscript is the authors' original work and has not been published nor has it been submitted simultaneously elsewhere. All authors have checked the manuscript and have agreed to the submission.

b. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Key Research and Development Project of State Grid Jiangxi Electric Power Company (Grant No. 52183524000).

Data availability

Data will be made available on request.

References

[1] J. Konečný, H.B. McMahan, D. Ramage, P. Richtárik, Federated optimization: Distributed machine learning for on-device intelligence, 2016, arXiv preprint arXiv:1610.02527.

[2] H. Liang, Y. Li, C. Zhang, X. Liu, L. Zhu, EGIA: An external gradient inversion attack in federated learning, IEEE Trans. Inf. Forensics Secur. 18 (2023) 4984–4995.

[3] J. Shan, Z. Zhao, J. Lu, R. Zhang, S.M. Yiu, K.-H. Chow, Geminio: Language-guided gradient inversion attacks in federated learning, 2024, arXiv preprint arXiv:2411.14937.

[4] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, T. Baker, VeriFL: Communication-efficient and fast verifiable aggregation for federated learning, IEEE Trans. Inf. Forensics Secur. 16 (2021) 1736–1751.

[5] H. Xie, T. Zhou, S. Chen, Y. Li, X. Yang, VCFL: Verifiable and communication-efficient federated learning against collusion attack for secure aggregation, Inform. Sci. 717 (2025) 122324.

[6] L. Chen, D. Xiao, X. Xiao, Y. Zhang, Secure and efficient federated learning via novel authenticable multi-party computation and compressed sensing, IEEE Trans. Inf. Forensics Secur. 19 (2024) 10141–10156.

[7] H. Kaminaga, F.M. Awaysheh, S. Alawadi, L. Kamm, MPCFL: Towards multi-party computation for secure federated learning aggregation, in: Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing, ACM, 2023, pp. 1–10.

[8] T. Stevens, C. Skalka, C. Vincent, J. Ring, S. Clark, J. Near, Efficient differentially private secure aggregation for federated learning via hardness of learning with errors, in: 31st USENIX Security Symposium, USENIX Association, 2022, pp. 1379–1395.

[9] J. Ling, J. Zheng, J. Chen, Efficient federated learning privacy preservation method with heterogeneous differential privacy, Comput. Secur. 139 (2024) 103715.

[10] J. Hu, J. Du, Z. Wang, X. Pang, Y. Zhou, P. Sun, K. Ren, Does differential privacy really protect federated learning from gradient leakage attacks? IEEE Trans. Mob. Comput. 23 (12) (2024) 12635–12649.

[11] Y. Li, N. Yan, J. Chen, X. Wang, J. Hong, K. He, W. Wang, B. Li, FedPHE: A secure and efficient federated learning via packed homomorphic encryption, IEEE Trans. Dependable Secur. Comput. 22 (5) (2025) 5448–5463.

[12] Y. Li, J. Lai, R. Zhang, M. Sun, Secure and efficient multi-key aggregation for federated learning, Inform. Sci. 654 (2024) 119830.

[13] J. Ma, S.-A. Naas, S. Sigg, X. Lyu, Privacy-preserving federated learning based on multi-key homomorphic encryption, Int. J. Intell. Syst. 37 (9) (2022) 5880–5901.

[14] E. Hosseini, A. Khisti, Secure aggregation in federated learning via multiparty homomorphic encryption, in: 2021 IEEE Globecom Workshops (GC Wkshps), IEEE, 2021, pp. 1–6.

[15] H. Chen, W. Dai, M. Kim, Y. Song, Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2019, pp. 395–412.

[16] Microsoft SEAL (release 4.1), 2023, [Online]. Available: https://github.com/Microsoft/SEAL.

[17] J. Wu, W. Zhang, F. Luo, ESAFL: Efficient secure additively homomorphic encryption for cross-silo federated learning, IEEE Trans. Dependable Secur. Comput. 22 (4) (2025) 3868–3884.

[18] J.H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: Advances in Cryptology – ASIACRYPT 2017, Springer, 2017, pp. 409–437.

[19] Z. Brakerski, Fully homomorphic encryption without modulus switching from classical GapSVP, in: Advances in Cryptology – CRYPTO 2012, Springer, 2012, pp. 868–886.

[20] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, Z. Wan, SecFed: A secure and efficient federated learning based on multi-key homomorphic encryption, IEEE Trans. Dependable Secur. Comput. 21 (4) (2024) 3817–3833.

[21] H. Tian, F. Zhang, Y. Shao, B. Li, Secure linear aggregation using decentralized threshold additive homomorphic encryption for federated learning, 2021, arXiv preprint arXiv:2111.10753.

[22] S. Hohenberger, A. Lysyanskaya, How to securely outsource cryptographic computations, in: Theory of Cryptography, Vol. 3378, Springer, 2005, pp. 264–282.

[23] O. Torki, H. Mala, M. Ashouri-Talouki, Secure fully-verifiable outsourcing of modular exponentiation: two servers check each other, J. Supercomput. 81 (1) (2025) 296.

[24] Q. Hu, M. Duan, Z. Yang, S. Yu, B. Xiao, Efficient parallel secure outsourcing of modular exponentiation to cloud for IoT applications, IEEE Internet Things J. 8 (16) (2021) 12782–12791.

[25] C. Tian, J. Yu, H. Zhang, H. Xue, C. Wang, K. Ren, Novel secure outsourcing of modular inversion for arbitrary and variable modulus, IEEE Trans. Serv. Comput. 15 (1) (2022) 241–253.

[26] Y. Ping, X. Guo, B. Wang, J. Zhou, Secure outsourcing of modular inverses and scalar multiplications on elliptic curves, Int. J. Secur. Netw. 15 (2) (2020) 101.

[27] H. Zhang, L. Tong, J. Yu, J. Lin, Blockchain-aided privacy-preserving outsourcing algorithms of bilinear pairings for internet of things devices, IEEE Internet Things J. 8 (20) (2021) 15596–15607.

[28] J. Gao, Y. Gao, GMS: an efficient fully homomorphic encryption scheme for secure outsourced matrix multiplication, J. Supercomput. 80 (18) (2024) 26435–26461.

[29] X. Ma, C. Li, Y. Hu, Y. Tao, Y. Jiang, Y. Li, F. Kong, C. Ge, Secure outsourced decryption for FHE-based privacy-preserving cloud computing, J. Inf. Secur. Appl. 86 (2024) 103893.

[30] X. Li, R. Li, B. Bai, Y. Zhao, G. Liu, R. Li, Outsourcing the computation of plaintext encryption for homomorphic encryption, in: 2023 8th International Conference on Computer and Communication Systems (ICCCS), IEEE, 2023, pp. 408–413.

[31] H.B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A.y. Arcas, Communication-efficient learning of deep networks from decentralized data, 2016, arXiv preprint arXiv:1602.05629.

[32] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: Advances in Cryptology — EUROCRYPT '99, Springer, 1999, pp. 223–238.

[33] X. Hu, J. Li, C. Wei, W. Li, X. Zeng, P. Yu, H. Guan, STYX: A hierarchical key management system for elastic content delivery networks on public clouds, IEEE Trans. Dependable Secur. Comput. 18 (2) (2021) 843–857.

[34] J. Wu, F. Luo, T. Sun, H. Wang, W. Zhang, Privacy-preserving federated learning scheme with mitigating model poisoning attacks: Vulnerabilities and countermeasures, IEEE Trans. Dependable Secur. Comput. 23 (1) (2026) 1421–1438.

[35] G. Qiu, Y. Zhao, X. Gui, Efficient privacy-preserving outsourced k-means clustering on distributed data, Inform. Sci. 674 (2024) 120687.

[36] O. Goldreich, Foundations of Cryptography: Volume 2, Basic Applications, Cambridge University Press, 2004.
[37] V. Lyubashevsky, C. Peikert, O. Regev, On ideal lattices and learning with errors over rings, in: Advances in Cryptology – EUROCRYPT 2010, Springer Berlin Heidelberg, 2010, pp. 1–23.
[38] HEAAN library, 2017, [Online]. Available: https://github.com/kimandrik/HEAAN/.
[39] NTL library (release 11.6.0), 2025, [Online]. Available: https://libntl.org/.
[40] GMP library (release 6.3.0), 2023, [Online]. Available: https://gmplib.org/.
[41] J.-P. Bossuat, R. Cammarota, I. Chillotti, B.R. Curtis, W. Dai, H. Gong, E. Hales, D. Kim, B. Kumara, C. Lee, X. Lu, C. Maple, A. Pedrouzo-Ulloa, R. Player, Y. Polyakov, L.A.R. Lopez, Y. Song, D. Yhee, Security guidelines for implementing homomorphic encryption, IACR Commun. Cryptol. 1 (4) (2025).
[42] M.R. Albrecht, R. Player, S. Scott, On the concrete hardness of learning with errors, J. Math. Cryptol. 9 (3) (2015) 169–203.
[43] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, Y. Liu, BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning, in: Proceedings of the 2020 USENIX Annual Technical Conference, USENIX Association, 2020, pp. 493–506.
[44] G.N. Gunesli, M. Bilal, S.E.A. Raza, N.M. Rajpoot, FedDropoutAvg: Generalizable federated learning for histopathology image classification, 2021, arXiv preprint arXiv:2111.13230.
[45] S. Caldas, J. Konečný, H.B. McMahan, A. Talwalkar, Expanding the reach of federated learning by reducing client resource requirements, 2018, arXiv preprint arXiv:1812.07210.